

Relatório Técnico
Computação Paralela e Distribuída
2025/2026
Licenciatura em Eng^a. Informática



Docente: Aníbal Ponte

Grupo: PL4_G07

Filipe Patrício – 202300133

Diogo Guedes – 202300686

Índice

Introdução.....	2
Contextualização	2
Objetivos gerais	2
Descrição da Solução.....	2
Organização geral do código	3
Implementação Sequencial	3
Procura do maior número primo.....	4
Game of Life	4
Paralelização	4
Procura do maior número primo.....	5
Game of Life	5
Sistema Distribuído (RPC com Sockets).....	5
Arquitetura cliente-servidor	6
Protocolo de comunicação.....	6
Operações RPC disponíveis.....	6
Concorrência no servidor	6
Principais decisões de implementação.....	6
Análise de Desempenho	6
Procura do maior número primo.....	7
Game of Life	7
Análise Comparativa e Discussão	7
Procura do maior número primo.....	8
Game of Life	8
Conclusões gerais	8
Conclusão.....	8

Introdução

Contextualização

Este projeto teve como objetivo aprofundar os conhecimentos e competências adquiridos na unidade curricular de Computação Paralela e Distribuída, através do desenvolvimento de soluções práticas que cobrem os principais temas abordados ao longo do semestre.

Foram definidas três grandes componentes de trabalho:

- A procura do maior número primo dentro de um limite temporal
- A simulação do autómato de células *Game of Life*
- A comunicação entre cliente e servidor através de RPC implementado sobre sockets

Objetivos gerais

Numa primeira fase, foram desenvolvidas as soluções para a procura do maior número primo e para o *Game of Life*, em versões sequenciais e, mais tarde, paralelas.

Numa segunda fase, as funcionalidades implementadas foram disponibilizadas remotamente através de um sistema cliente-servidor, onde o cliente invoca as operações no servidor via RPC e recebe os respetivos resultados, em vez de estas serem executadas localmente com input direto.

Descrição da Solução

Organização geral do código

Os ficheiros do projeto estão organizados em:

- **Primos.py** : Implementação de algoritmos para verificação de primalidade (is_prime) e procura do maior número primo, com versões sequencial e paralela (multiprocessing). Utiliza wheel factorization base 6 como otimização.
- **GameofLife.py** : Simulação do autómato celular Game of Life seguindo as regras de Conway, com vizinhança Moore (8 vizinhos). Oferece versões sequencial e paralela com divisão da grelha por linhas para máxima eficiência de cache.
- **Servidor.py** : Servidor RPC (JSON-RPC simplificado) que escuta na porta 9000, aceitando ligações de múltiplos clientes de forma sequencial e expondo as funcionalidades através de introspeção dinâmica.
- **Cliente.py** : Cliente interativo em CLI que se conecta ao servidor via TCP socket, permitindo ao utilizador invocar operações e visualizar resultados em tempo real.
- **GoL_Client_GUI.py**: Cliente com interface gráfica dedicada para Game of Life, com visualização da evolução da grelha.
- **Testes.py** : Suite de testes automáticos em unittest para validação de funcionalidades, incluindo testes de correção, desempenho e comportamento paralelo.

Implementação Sequencial

Procura do maior número primo

A versão sequencial da procura do maior número primo começa no número 2 e percorre os números inteiros de forma crescente, testando cada um com a função `is_prime` dada no enunciado. O maior primo encontrado é atualizado ao longo da execução e devolvido quando o tempo limite é atingido.

A função `is_prime` utiliza um algoritmo de teste de divisibilidade otimizado — primeiro verifica primeiro os casos especiais (números menores que 2, e os primos 2 e 3), elimina de seguida os múltiplos de 2 e de 3, e testa depois apenas divisores da forma $6k \pm 1$ até $\sqrt{n}$. Esta abordagem reduz significativamente o número de divisões necessárias face a uma verificação exaustiva, com complexidade $O(\sqrt{n})$.

Game of Life

- **Fundamentos Teóricos:**

O Autómato Celular de Conway (Game of Life) é um sistema de evolução discreta onde cada célula numa grelha bidimensional pode estar em dois estados: viva (1) ou morta (0). A evolução ocorre em gerações sucessivas, cada uma determinada pelas regras de Conway aplicadas simultaneamente a todas as células.

Regras de Evolução:

As regras determinam o próximo estado de uma célula com base no estado atual e no número de vizinhos vivos adjacentes, utilizando a vizinhança de Moore (8 células adjacentes):

- 0 (Subpopulação): Se a célula está viva e tem menos de 2 vizinhos vivos.
- 1 (Mantém-se viva): Se a célula está viva e tem 2 ou 3 vizinhos vivos.
- 0 (Sobrepopulação): Se a célula está viva e tem mais de 3 vizinhos vivos.
- 1 (Reprodução): Se a célula está morta e tem exatamente 3 vizinhos vivos.
- 0 (Mantém-se morta): Caso contrário.

Estrutura de Dados:

Representação da Grelha: Lista de listas Python com valores inteiros 0 (morta) ou 1 (viva).

Vizinhança: Não-cíclica (grelha finita sem wrapping). Células nas fronteiras possuem menos de 8 vizinhos.

- **Versão sequencial:**

```
def game_of_life_sequential(grid, generations):  
    """  
    Simula a evolução durante um número fixo de gerações.  
  
    Args:  
        grid (list[list[int]]): Grelha inicial  
        generations (int): Número de gerações a simular  
  
    Returns:  
        list[list[int]]: Grelha após simulação  
    """  
    current_grid = deepcopy(grid)  
    num_rows = len(current_grid)  
    num_cols = len(current_grid[0])  
  
    for _ in range(generations):  
        # Criar nova grelha para próxima geração  
        next_grid = [[0 for _ in range(num_cols)] for _ in range(num_rows)]  
  
        # Aplicar regras a cada célula  
        for row in range(num_rows):  
            for col in range(num_cols):  
                num_neighbors = count_neighbors(current_grid, row, col)  
                next_grid[row][col] = apply_rules(current_grid, row, col, num_neighbors)  
  
        # Atualizar para próxima iteração  
        current_grid = next_grid  
  
    return current_grid
```

- **Principais decisões de implementação:**

- **Hipótese 1: Duas Grelhas com Cópia Profunda (Implementada)**

Descrição: Esta abordagem utiliza duas grelhas distintas em memória: uma de leitura (grelha atual) e outra de escrita (próxima grelha). O estado futuro de todas as células é calculado com base no estado estático da geração anterior.

Características: Utiliza duas grelhas independentes, garantindo a simultaneidade clássica do algoritmo. A memória ocupada é o dobro do tamanho da matriz ($2 \times O(R \times C)$). O input original é preservado através de uma cópia profunda (deepcopy). A complexidade temporal é de $O(G \times R \times C)$, com um fator de overhead controlado.

Vantagens: Garante a correção biológica do Jogo da Vida (as células evoluem em simultâneo); o código é legível e puro (sem efeitos colaterais); é fácil de debugar e de testar contra outputs esperados.

Desvantagens: Duplica o consumo de memória RAM e introduz o overhead de alocação e cópia de estruturas (cerca de 1 a 2 ms para matrizes de 500x500).

Justificação de Escolha: Foi a hipótese selecionada porque a correção algorítmica é prioritária face à performance absoluta num projeto educacional.

```
def game_of_life_sequential(grid, generations):
    current_grid = deepcopy(grid) # Cópia profunda

    for gen in range(generations):
        next_grid = [[0] * len(current_grid[0]) for _ in range(len(current_grid))]

        for row in range(len(current_grid)):
            for col in range(len(current_grid[0])):
                neighbors = count_neighbors(current_grid, row, col)
                next_grid[row][col] = apply_rules(current_grid, row, col, neighbors)

        current_grid = next_grid

    return current_grid
```

- **Hipotese 2: Modificação In-Place com Sincronização Manual**

Descrição: Tentativa de atualizar os estados das células diretamente na grelha original à medida que esta é percorrida, com o objetivo de poupar memória.

Características: Utiliza apenas uma grelha em memória ($O(R \times C)$), modificando o input original diretamente. A sincronização falha porque o cálculo de uma célula passa a ler valores que já foram alterados na mesma iteração.

Vantagens: Alta eficiência de memória (redução de 50% de RAM) e execução ligeiramente mais rápida por evitar alocações.

Desvantagens: Ocorre uma violação crítica do princípio de simultaneidade. O resultado torna-se incorreto, perde o determinismo e o comportamento deixa de representar as regras reais de Conway.

Razão da não Implementação: Falha fundamental. Sacrificar a correção biológica por um ganho marginal de performance não é aceitável, pois desvirtua o propósito do algoritmo.

```
def game_of_life_inplace(grid, generations):
    """Tentativa de modificar in-place com threads."""

    for gen in range(generations):
        # ERRADO: Modificar durante iteração
        for row in range(len(grid)):
            for col in range(len(grid[0])):
                neighbors = count_neighbors(grid, row, col)
                # X PROBLEMA: Modifica enquanto ainda está a processar
                grid[row][col] = apply_rules(grid, row, col, neighbors)

    return grid
```

- **Hipótese 3: Grelha Cíclica - Toro**

Descrição: Implementação onde as fronteiras da grelha estão interligadas (efeito de wrapping ou Pac-Man), transformando a matriz bidimensional num toro matemático tridimensional.

Características: Não existem células de fronteira especiais; todas as células possuem sempre exatamente 8 vizinhos através do operador aritmético de resto da divisão inteira (módulo).

Vantagens: Simetria matemática perfeita e ausência de efeitos de borda morta, permitindo que estruturas como os gliders circulem infinitamente pela grelha.

Desvantagens: Comportamento visualmente contra-intuitivo para o utilizador comum. Além disso, o cálculo contínuo do operador módulo em cada célula adiciona um overhead de processamento de 5% a 10%.

Razão da não Implementação: Afasta-se da especificação padrão definida por John Conway, que dita uma grelha finita com fronteiras estáticas. Adiciona complexidade computacional e de validação sem benefícios claros para o escopo do projeto.

```
def game_of_life_cyclic(grid, generations):
    """Grelha cíclica (sem fronteiras)."""

    for gen in range(generations):
        next_grid = [[0] * len(grid[0]) for _ in range(len(grid))]

        for row in range(len(grid)):
            for col in range(len(grid[0])):
                # Vizinhos com wrapping (modulo)
                neighbors = 0
                for dr in [-1, 0, 1]:
                    for dc in [-1, 0, 1]:
                        if dr == 0 and dc == 0:
                            continue
                        nr = (row + dr) % len(grid)      # ← Modulo wrapping
                        nc = (col + dc) % len(grid[0])    # ← Modulo wrapping
                        neighbors += grid[nr][nc]

                next_grid[row][col] = apply_rules(grid, row, col, neighbors)

        grid = next_grid

    return grid
```

- **Decisão Final e Conhecimento Obtido**

A Hipótese 1 (Duas Grelhas com Cópia Profunda) foi escolhida unanimemente por maximizar os critérios de correção, clareza, testabilidade e compatibilidade com o padrão académico estabelecido.

Desta análise, retiram-se duas grandes lições para o desenvolvimento do projeto:

1. **Prioridade Absoluta:** Código legível e matematicamente correto é sempre preferível a código otimizado que produz outputs errados.
2. **Preparação para o Futuro:** A utilização de funções puras na Hipótese 1 (onde o estado atual nunca é alterado enquanto se escreve no estado futuro) cria a fundação ideal para isolar dados, facilitando a posterior paralelização estável do algoritmo.

Paralelização

Procura do maior número primo

Alternativas consideradas

A primeira abordagem implementada consistia em cada worker começar num número muito elevado (10^{15}) e avançar com saltos fixos grandes entre posições, sem qualquer divisão por intervalos. Embora esta estratégia permitisse explorar números de maior dimensão, revelou-se pouco eficiente pois não havia garantia de cobertura ordenada do espaço de procura e o maior primo encontrado dependia essencialmente da sorte da distribuição inicial.

Após análise de desempenho e discussão com o docente, foi adotada uma abordagem baseada em intervalos, descrita em detalhe abaixo.

Estratégia de divisão do espaço de procura (Estratégia final)

O espaço de procura é dividido em intervalos contíguos de tamanho fixo (`INTERVAL_SIZE =  $10^6$`). Cada worker fica responsável pelos intervalos cujo índice segue o padrão `worker_id + k * num_workers`, garantindo cobertura sem sobreposição. Dentro de cada intervalo, o worker

começa pelo valor mais alto e desce, o que garante que o primeiro primo encontrado é necessariamente o maior desse intervalo — permitindo parar imediatamente e avançar para o próximo intervalo.

A escolha do valor 10^6 resultou de uma análise experimental. Valores maiores como 10^{16} ou 10^{18} foram testados e mostraram que o tempo de execução real excedia significativamente o timeout definido — por exemplo, com 10^{18} o programa demorou cerca de 102 segundos para um timeout de 30 segundos. Isto deve-se ao facto de `is_prime(n)` ter complexidade $O(\sqrt{n})$, pelo que números muito grandes tornam cada verificação demorada e o `stop_event` deixa de ser verificado com frequência suficiente. Com 10^6 , cada verificação é praticamente instantânea (divisores até $\sqrt{10^6} = 1000$) e o timeout é respeitado com uma margem inferior a 1 segundo.

Porquê processos e não threads

A decisão de utilizar processos (multiprocessing) em vez de threads foi tomada desde o início do desenvolvimento. Em Python, o GIL (*Global Interpreter Lock*) impede que múltiplas threads executem código Python simultaneamente, o que na prática elimina o paralelismo real em tarefas CPU-intensivas como a verificação de primalidade. Os processos têm memória própria e correm em núcleos distintos, contornando esta limitação e permitindo tirar partido dos 16 cores disponíveis na máquina utilizada.

Mecanismos de sincronização

Foram utilizados três mecanismos de sincronização partilhados entre processos:

- `Value('Q', 2)` — variável inteira de 64 bits partilhada entre processos, que mantém o maior primo encontrado até ao momento.
- `Lock` — garante que as atualizações são atómicas, evitando condições de corrida.

- **Event** — sinaliza a paragem coordenada de todos os workers quando o timeout é atingido. Cada worker verifica este evento no início de cada iteração e termina de forma ordenada.

Dificuldades encontradas no processo

A principal dificuldade foi chegar a uma estratégia de divisão do espaço de procura que fosse simultaneamente eficiente e que respeitasse o timeout definido. A abordagem inicial com números muito grandes mostrou que existe um compromisso direto entre a dimensão dos números testados e o tempo de cada verificação — quanto maior o número, mais lenta é a verificação e menos controlo existe sobre o tempo de execução real.

Game of Life

- **Alternativas consideradas**

Durante a fase de desenho da solução paralela, foram avaliadas três abordagens principais para gerir o estado e a concorrência em Python:

- **MultiThreading:**

Descartado devido ao GIL (Global Interpreter Lock) do Python. Como a simulação do Game of Life envolve computação intensiva de CPU (loops aninhados para contar vizinhos e aplicar regras), múltiplas threads partilhando o mesmo interpretador não trariam ganhos reais de performance, correndo sequencialmente na prática.

- **Partilha de memória com Locks:**

Considerou-se manter uma única grelha global em memória partilhada estruturada, onde cada processo lia os vizinhos e atualizava a sua secção recorrendo a trincas (locks) ou barreiras. Contudo, a sobrecarga de sincronização (overhead) e a

complexidade para evitar condições de corrida (race conditions) tornavam o código propenso a erros.

- **Abordagem com Process Pool (escolhida):**

Optou-se por isolar os dados criando processos trabalhadores (workers) independentes. O processo pai distribui cópias das fatias necessárias da grelha e recolhe as respostas. Esta abordagem mitiga o GIL por usar processos separados e simplifica o fluxo de dados, sendo a mais robusta em Python standard.

- **Estratégia de divisão da grelha**

A estratégia adotada assenta numa decomposição unidimensional por linhas (fatiamento horizontal) através da função (divide_grid_into_regions).

- **Distribuição Equitativa:**

O número total de linhas da grelha é dividido pelo número de workers ativos ($\text{rows_per_worker} = \text{num_rows} // \text{effective_workers}$).

- **Gestão do Resto:**

Se o número de linhas não for perfeitamente divisível pelos workers, o resto da divisão (remaining_rows) é distribuído uniformemente adicionando uma linha extra aos primeiros workers. Isto garante um balanceamento de carga (load balancing) otimizado, evitando que um único processo fique com uma sobrecarga desproporcional.

- **Gestão das fronteiras entre regiões**

- **Abordagem por Cópia Global:**

Em vez de enviar apenas pedaços isolados e gerir ghost cells (linhas fantasma de comunicação), a tua implementação resolve isto enviando a grelha completa atualizada para cada worker em cada geração ($\text{worker_args} = [(\text{current_grid}, \text{start_row}, \text{end_row}) \dots]$).

- **Isolamento no Cálculo:**

O worker recebe a grelha toda (grid), o que significa que quando a função `count_neighbors` é chamada para uma linha limite (ex: `start_row`), ela consegue aceder nativamente à linha de cima (`start_row - 1`) porque a estrutura global está presente.

- **Escrita Local:**

O worker apenas calcula e guarda os novos estados no seu subset relativo (`region[row - start_row][col]`), garantindo que nenhuma fronteira é alterada prematuramente antes de todos os processos terminarem a leitura.

- **Mecanismos de sincronização**

A simulação do Game of Life é um algoritmo síncrono por gerações: a geração N+1 não pode começar a ser calculada até que todas as células da geração N estejam processadas.

- **Sincronização Implícita via (`pool.map()`):**

O código utiliza o método `pool.map(_compute_region, worker_args)`. Este método é bloqueante, funcionando como uma barreira de sincronização natural. O processo principal submete os pedaços para os workers e aguarda obrigatoriamente que todos concluam as suas respectivas regiões.

- **Reconstrução Sequencial:**

Assim que o `pool.map` desbloqueia, os resultados parciais retornam ordenados. O processo pai faz a concatenação das regiões (`next_grid.extend(region)`) numa única estrutura centralizada, avançando o relógio da simulação para a iteração seguinte.

- **Dificuldades encontradas**

Embora a lógica paralela esteja correta e produza resultados idênticos à sequencial, a implementação enfrenta algumas dificuldades e limitações de performance típicas de Python:

- **Overhead de criação de processos:**

Nas funções `game_of_life_parallel` e `_timeout`, o bloco `with mp.Pool(...)` está dentro do loop das gerações. Isto significa que a cada geração o Python cria, inicializa e destrói um Pool de processos. O custo temporal de criar processos no sistema operativo é massivo e acaba por anular o ganho da paralelização (gerando speedups inferiores a 1 em grelhas pequenas/médias).

- **Custo de Serialização (IPC):**

Passar a `current_grid` completa como argumento para cada worker via `worker_args` obriga o Python a serializar (via pickle) a matriz em memória e enviá-la por canais de comunicação entre processos (IPC). Para matrizes muito grandes, este tráfego de dados torna-se o principal gargalo da aplicação.

- **Granularidade Fina:**

O Game of Life realiza computações muito simples por célula (operações matemáticas elementares). O rácio entre o tempo gasto a processar e o tempo gasto a gerir a paralelização (comunicação/sincronização) é desfavorável, exigindo matrizes gigantescas para que a versão paralela compense a sequencial.

Sistema Distribuído (RPC com Sockets)

Arquitetura cliente-servidor

O sistema segue uma arquitetura cliente-servidor clássica. O servidor corre continuamente à escuta de ligações TCP na porta 9000, expondo as funcionalidades implementadas na Componente 1. O cliente liga-se ao servidor, invoca operações remotamente e apresenta os resultados ao utilizador. A comunicação foi testada em localhost (127.0.0.1), simulando um ambiente distribuído na mesma máquina.

Foi também desenvolvida, a pedido do docente, uma interface gráfica em tkinter (GoL_Client_GUI.py) que permite visualizar a evolução do *Game of Life* em tempo real, integrando as mesmas chamadas RPC do cliente de texto.

Protocolo de comunicação

A comunicação segue um modelo pedido-resposta (*request-response*) sobre TCP, com mensagens no formato JSON.

Formato do pedido:

```
{"method": "nome_da_funcao", "params": {"param1": valor}}
```

Formato da resposta em caso de sucesso:

```
{"result": valor}
```

Formato da resposta em caso de erro:

```
{"error": "mensagem de erro"}
```

Uma decisão de implementação foi a delimitação de mensagens com o carácter `\n` no fim de cada JSON enviado. O TCP é um protocolo de stream — não garante que uma mensagem chegue inteira numa única leitura. Sem delimitador, o recetor não saberia onde uma mensagem terminaria e a seguinte começa. A solução adotada foi acumular os bytes recebidos até encontrar o `\n`, só depois é que desserializa o JSON.

O tratamento de erros cobre os seguintes casos: campo `method` ausente, campo `params` ausente ou inválido, método inexistente, parâmetros em falta, e erros internos durante a execução da função.

Operações RPC disponíveis

```
Cliente RPC – Números Primos + Game of Life

[1] Listar métodos disponíveis
[2] Verificar se número é primo
[3] Encontrar maior primo - Sequencial
[4] Encontrar maior primo - Paralelo
[5] Game of Life - Simulação sequencial
[6] Game of Life - Simulação paralela
[7] Game of Life - Interface Gráfica
[0] Sair
```

Principais decisões de implementação

A decisão de usar threading no servidor (em vez de multiprocessing) foi consciente — o servidor apenas coordena pedidos e delega o trabalho pesado para as funções da Componente 1, que já usam multiprocessing internamente. Usar processos no servidor adicionaria complexidade desnecessária sem benefício prático.

O dicionário METHODS centraliza o registo de todas as funções expostas, tornando simples adicionar novos métodos — basta inserir uma entrada no dicionário sem alterar a lógica de handle_request.

Análise de Desempenho

Procura do maior número primo

Abordagem Sequencial vs Paralelo para 60s:

Sequencial

```
Tempo limite (segundos): 60
A procurar (sequencial) durante 60s... (aguarde)

Maior primo encontrado : 11384677
Notação científica      : 1.138468e+07
Número de dígitos       : 8
```

Paralelo com 16 workers (testado numa maquina de 16 cores)

```
Tempo limite (segundos): 60
Número de workers: 16
A procurar (paralelo com 16 workers) durante 60s... (aguarde)

Maior primo encontrado : 936999999999983
Notação científica      : 9.370000e+14
Número de dígitos       : 15
```

Análise da abordagem paralela para vários workers:

1 worker

```
Tempo limite (segundos): 60
Número de workers: 1
A procurar (paralelo com 1 workers) durante 60s... (aguarde)

Maior primo encontrado : 188999999999983
Notação científica      : 1.890000e+14 ←
Número de dígitos       : 15
```

4 workers

```
Tempo limite (segundos): 60
Número de workers: 4
A procurar (paralelo com 4 workers) durante 60s... (aguarde)

Maior primo encontrado : 469999999999991
Notação científica      : 4.700000e+14 ←
Número de dígitos       : 15
```

16 workers

```
Tempo limite (segundos): 60
Número de workers: 16
A procurar (paralelo com 16 workers) durante 60s... (aguarde)

Maior primo encontrado : 989999999999993
Notação científica      : 9.900000e+14 ←
Número de dígitos       : 15
```

Maior primo em 60 s (1 min): 9.9×10^{14}

Maior primo em 120 s (2 min): 1.4×10^{15}

Maior primo em 240 s (4 min): 2.2×10^{15}

Game of Life

- **Sequencial vs paralelo**

A análise comparativa entre as duas abordagens revela um comportamento contraintuitivo à primeira vista, mas perfeitamente justificável à luz da arquitetura da aplicação.

```
Cliente RPC – Números Primos + Game of Life

[1] Listar métodos disponíveis
[2] Verificar se número é primo
[3] Encontrar maior primo - Sequencial
[4] Encontrar maior primo - Paralelo
[5] Game of Life - Simulação sequencial
[6] Game of Life - Simulação paralela
[7] Game of Life - Interface Gráfica
[0] Sair

Opção: 5
Linhas do grid (50): 50
Colunas do grid (50): 50
Tempo limite (segundos): 5
A simular (50x50) durante 5s... (aguarde)

Simulação sequencial concluída
Gerações executadas: 1961
Tempo limite: 5s
```

```
Cliente RPC – Números Primos + Game of Life

[1] Listar métodos disponíveis
[2] Verificar se número é primo
[3] Encontrar maior primo - Sequencial
[4] Encontrar maior primo - Paralelo
[5] Game of Life - Simulação sequencial
[6] Game of Life - Simulação paralela
[7] Game of Life - Interface Gráfica
[0] Sair

Linhas do grid (50): 50
Colunas do grid (50): 50
Tempo limite (segundos): 5
Número de workers (4): 4
A simular (50x50) com 4 workers durante 5s... (aguarde)

Simulação paralela concluída
Workers: 4
Gerações executadas: 39
Tempo limite: 5s
```

- **Abordagem Sequencial (Vencedora):**
Conseguiu processar 2022 gerações dentro do tempo limite de 5 segundos.
- **Abordagem Paralela (4 workers):**
Processou apenas 39 gerações no mesmo intervalo de 5 segundos, atingindo uma média a rondar as 7,6 gerações por segundo.
- **Análise Crítica do Fenómeno (Overhead)**

O facto de a versão sequencial ter sido cerca de 53 vezes mais rápida que a versão paralela com 4 workers deve-se a três fatores críticos.

- **Granularidade Fina vs Custo de IPC:**
Uma grelha de 50 x 50 (2500 células) é considerada pequena para computação paralela. O cálculo matemático de vizinhos para 2500 células é extremamente rápido na CPU. O tempo que o Python gasta a serializar a matriz (via pickle) e a transmiti-la entre processos através de IPC (Inter-Process Communication) é ordens de magnitude superior ao tempo de cálculo propriamente dito.
- **Recriação do Pool de Processos:**
Como o bloco with mp.Pool (processes=...) está inserido dentro do ciclo while do timeout, a aplicação sofre a penalização de criar, inicializar e destruir 4 processos do sistema operativo a cada geração. O custo de ciclo de vida de processos destrói completamente o rendimento da paralelização.
- **Latência de Comunicação RPC:**
Como este teste foi executado através de um cliente RPC, o processamento paralelo acrescenta latência de gestão de concorrência num ambiente que já possui o overhead da camada de rede/comunicação remota.

Em suma, a paralelização nesta configuração introduziu um gargalo de gestão (overhead) que superou largamente o benefício da divisão de trabalho. A abordagem paralela só apresentaria vantagens competitivas em grelhas massivas (ex: 2000 x 2000), onde o volume de processamento justificasse o custo de comunicação.

Análise Comparativa

Procura do maior número primo

Eficiência da abordagem de intervalos

A abordagem de divisão por intervalos de tamanho 10^6 revelou-se eficiente para garantir simultaneamente a qualidade dos resultados e o respeito pelo timeout definido. Ao começar pelo fim de cada intervalo e descer até encontrar o primeiro primo — que é necessariamente o maior desse intervalo — cada worker evita processar números desnecessários dentro do intervalo, avançando imediatamente para o seguinte após encontrar um primo.

A escolha do tamanho do intervalo (10^6) resultou de análise experimental: intervalos maiores (10^{16} , 10^{18}) foram testados e causaram excesso significativo do tempo de execução relativamente ao timeout definido, devido à complexidade $O(\sqrt{n})$ da função `is_prime` para números muito grandes.

Escalabilidade com o número de workers:

Workers	Maior primo encontrado	Valor aproximado
1	188 999 999 999 983	1.89×10^{14}
4	469 999 999 999 991	4.70×10^{14}
16	989 999 999 999 993	9.90×10^{14}

com 16 workers não se obtém x16 o resultado de 1 worker. Isto deve-se principalmente ao overhead de criação e sincronização dos processos e ao facto de os intervalos mais avançados conterem números maiores, cuja verificação de primalidade ser mais demorada.

Foi também testado o impacto do timeout no resultado com 16 workers:

Timeout	Maior primo encontrado
60s	$\sim 9.9 \times 10^{14}$
120s	$\sim 1.4 \times 10^{15}$
240s	$\sim 2.2 \times 10^{15}$

O crescimento do resultado com o tempo é aproximadamente linear, o que indica que o sistema escala de forma consistente com o tempo disponível.

Limitações encontradas:

A principal limitação da abordagem é que o speedup não é linear com o número de workers. Para além do overhead de sincronização, à medida que os workers avançam para intervalos com números maiores, cada verificação de primalidade torna-se progressivamente mais lenta — o que significa que os workers mais avançados processam menos intervalos por segundo do que os iniciais.

Game of Life

- **Eficiência da divisão da grelha**
 - **Balanceamento de Carga (Load Balancing):**
A função `_divide_grid_into_regions` é matematicamente eficiente. Ao distribuir o resto das divisões (`remaining_rows`) pelas primeiras fatias, o algoritmo garante que a diferença máxima de carga entre o worker mais sobrecarregado e o menos sobrecarregado seja de apenas 1 linha.
 - **Fatiamento Unidimensional vs Bidimensional:**
O fatiamento horizontal (por linhas) é simples de implementar, mas introduz uma ineficiência geométrica. Cada região partilha uma fronteira linear completa com as regiões adjacentes. Para matrizes quadradas muito grandes, uma divisão em blocos bidimensionais (sub-grelhas em matriz) reduziria o perímetro de contacto e, consequentemente, o volume de dados de fronteira a sincronizar.
 - **Desperdício por Cópia Redundante:**
O maior fator de ineficiência nesta implementação específica não está na divisão em si, mas no facto de enviar a grelha completa para todos os workers. Se testarmos

com 4 workers, a memória envia 4 x grelha_completa em vez de enviar apenas a fatia correspondente mais as duas linhas de fronteira (ghost cells). Isto transforma uma operação que devia ser local num problema de saturação de memória.

- **Escalabilidade**

- **Escalabilidade Fortemente Negativa (Strong Scaling):**

Ao fazer os testes com uma grelha de 50 x 50, a escalabilidade foi severamente negativa. Ao passar de 1 processo (sequencial) para 4 processos, o rendimento colapsou de 2022 para 38 gerações. Isto demonstra que o sistema atingiu o chamado gargalo de Amdahl, onde o custo sequencial de gestão domina completamente a aplicação.

- **Escalabilidade Fraca:**

O algoritmo só demonstrará escalabilidade positiva se o tamanho do problema crescer substancialmente. Se ao testar uma matriz de 2000 x 2000, o custo de calcular as regras começa a ser tão massivo que compensará pagar a taxa de criação dos processos, permitindo ver curvas de aceleração (speedup) positivas com 2, 4 ou mais workers.

- **Limitações encontradas**

- **Ciclo de Vida do Pool Intraloop (A Maior Limitação):**

A função paralela cria e destrói o mp.Pool dentro do loop de tempo/gerações. Em sistemas operativos modernos, pedir novos processos ao kernel a cada escassos milissegundos gera uma sobrecarga catastrófica. O correto seria criar o Pool uma única vez no início da função e reutilizá-lo (ou usar primitivas de sincronização como mp.Barrier).

- **Abuso de Memória e Custo de Serialização (Pickling):**

O módulo multiprocessing do Python não partilha memória nativamente; ele copia dados entre processos isolados. Ao enviar a matriz inteira repetidamente, o Python passa mais tempo a converter listas em bytes (pickling) e a enviá-las pelos canais de comunicação do sistema (IPC) do que a executar a simulação.

- **Granularidade Demasiado Fina para Python:**

O Game of Life possui uma granularidade extremamente fina (pouquíssimas operações de CPU por cada byte de dado lido). O Python, sendo uma linguagem interpretada, já possui um custo nativo por loop. Quando misturado com

concorrência baseada em processos pesados, o modelo de arquitetura entra em colapso de eficiência para problemas de pequena e média escala.

- **Falta de Memória Partilhada Real:**

A ausência de estruturas como SharedMemory ou matrizes NumPy partilhadas, obriga a soluções de cópia profunda (deepcopy), limitando o algoritmo a estruturas de dados pesadas (listas de listas).

Conclusão

Este projeto permitiu consolidar os conhecimentos adquiridos ao longo do semestre nas áreas de computação paralela e distribuída, através da implementação prática de soluções para dois problemas computacionais distintos e da sua exposição remota via RPC.

Na Componente 1, foram desenvolvidas versões sequenciais e paralelas para a procura do maior número primo e para a simulação do *Game of Life*. Em ambos os casos, a paralelização revelou comportamentos distintos: na procura de primos, a versão paralela demonstrou ganhos claros com o aumento do número de workers, atingindo primos da ordem de 10^{14} a 10^{15} em 60 segundos com 16 workers. No *Game of Life*, a versão paralela revelou-se menos eficiente que a sequencial para grelhas de dimensão reduzida, devido ao overhead de criação de processos e serialização de dados — limitações inerentes ao modelo de multiprocessing em Python para problemas de granularidade fina.

Um aspeto importante que este projeto evidenciou é que paralelizar nem sempre significa melhorar o desempenho. A escolha do modelo de paralelização, do tamanho do problema e dos mecanismos de sincronização tem impacto direto nos resultados, e essas decisões devem ser fundamentadas em análise experimental, como foi feito ao longo deste trabalho.

Na Componente 2, foi implementado um sistema cliente-servidor funcional sobre sockets TCP, com comunicação em JSON e suporte a múltiplos clientes em simultâneo. A implementação cumpre os requisitos do enunciado sem recorrer a bibliotecas de RPC externas, e foi complementada com uma interface gráfica para o *Game of Life*, desenvolvida a pedido do docente.

Em suma, os objetivos propostos foram cumpridos, tendo o projeto contribuído para uma compreensão mais sólida dos desafios reais da computação paralela e distribuída, nomeadamente no que diz respeito ao equilíbrio entre complexidade de implementação, correção dos resultados e ganhos efetivos de desempenho.